

Convolutional Neural Network (CNN)

What is CNN?

A convolutional neural network (CNN or ConvNet) is a network architecture for deep learning that learns directly from data. A convolutional neural network is a feed-forward neural network that is generally used to analyze visual images by processing data with grid-like topology. A convolutional neural network is used to detect and classify objects in an image.

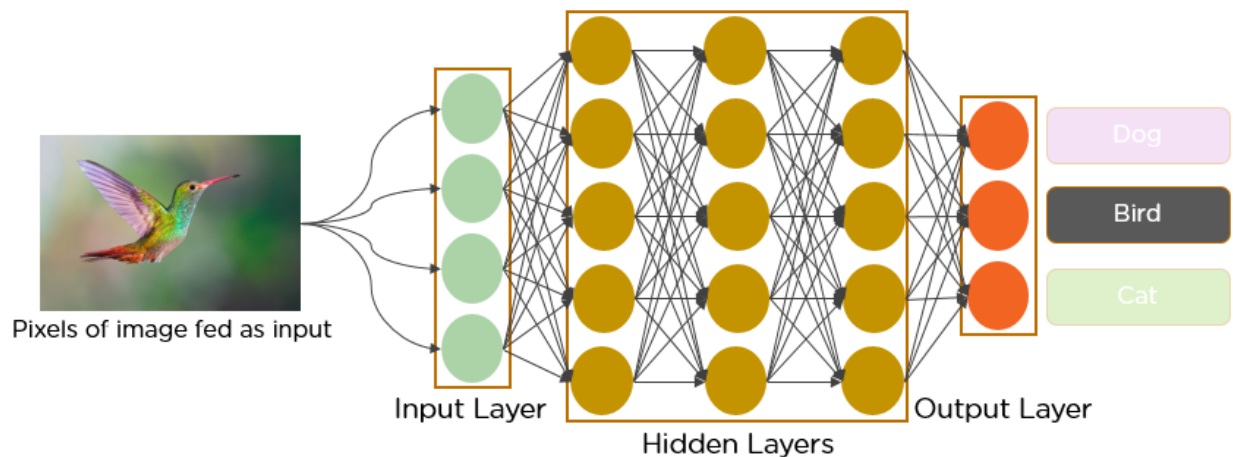
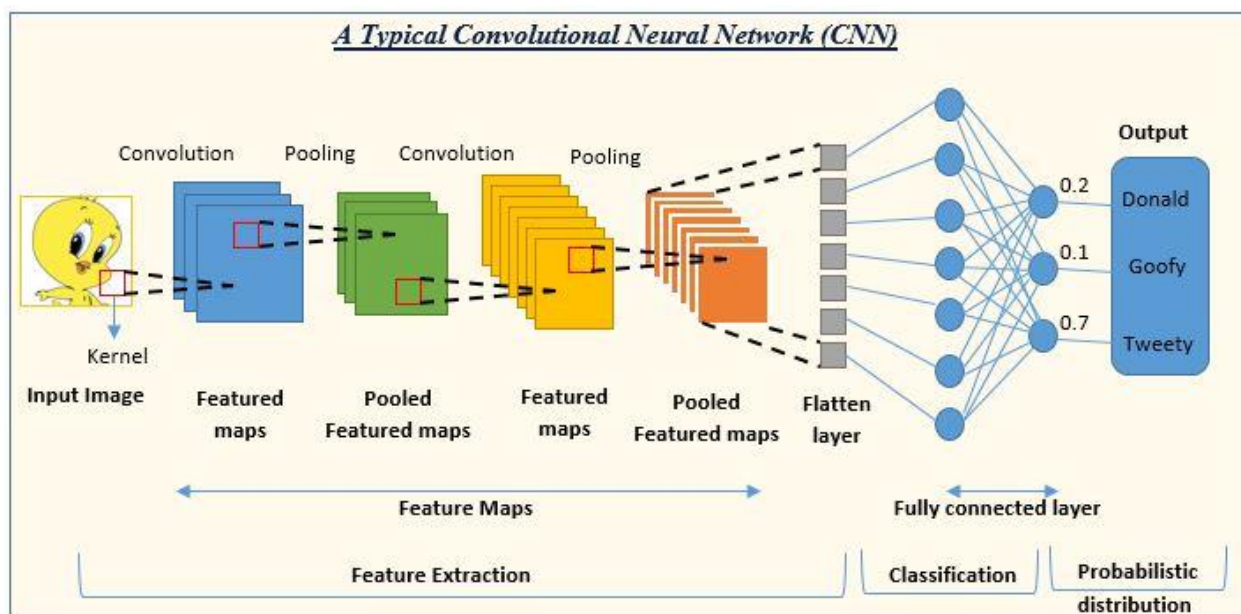


Fig: Convolutional Neural Network to identify the image of a bird

How CNNs work?

The pixels from the image are fed to the convolutional layer that performs the convolution operation. It results in a convolved map. The convolved map is applied to a ReLU function to generate a rectified feature map. The image is processed with multiple convolutions and ReLU layers for locating the features.

A CNN is composed of an input layer, an output layer, and many hidden layers in between. These layers perform operations that alter the data with the intent of learning features specific to the data. Three of the most common layers are convolution, activation or ReLU, and pooling.

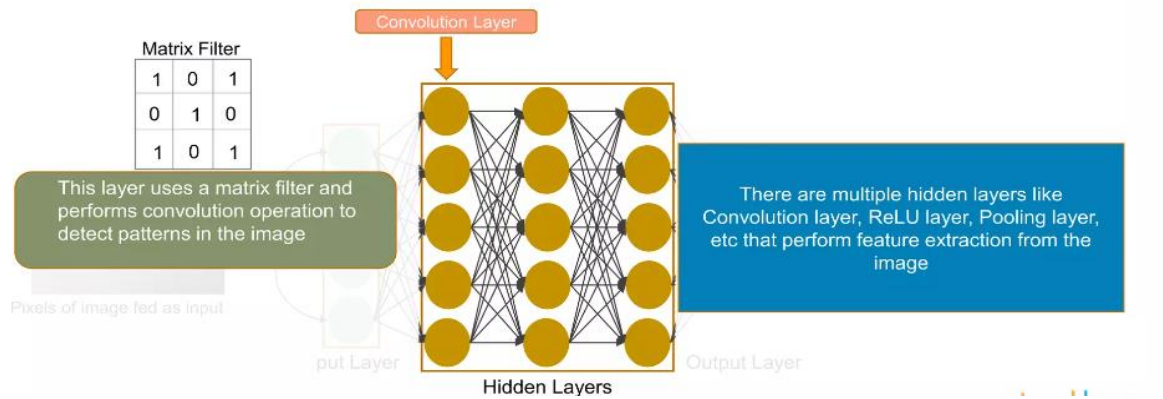
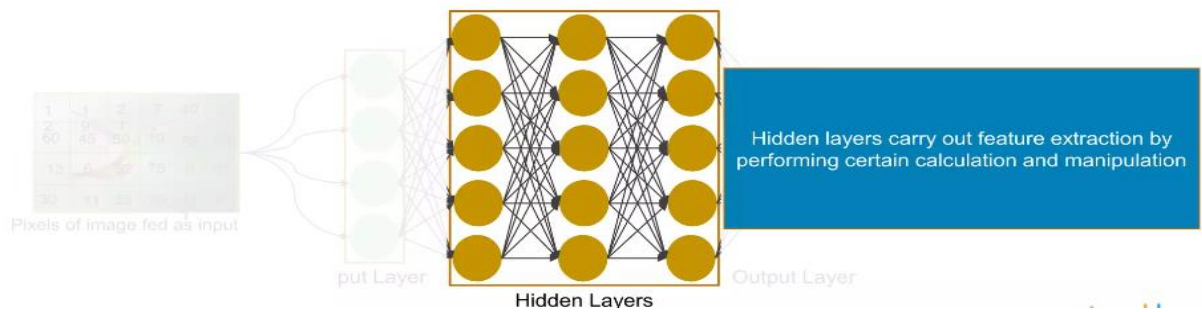
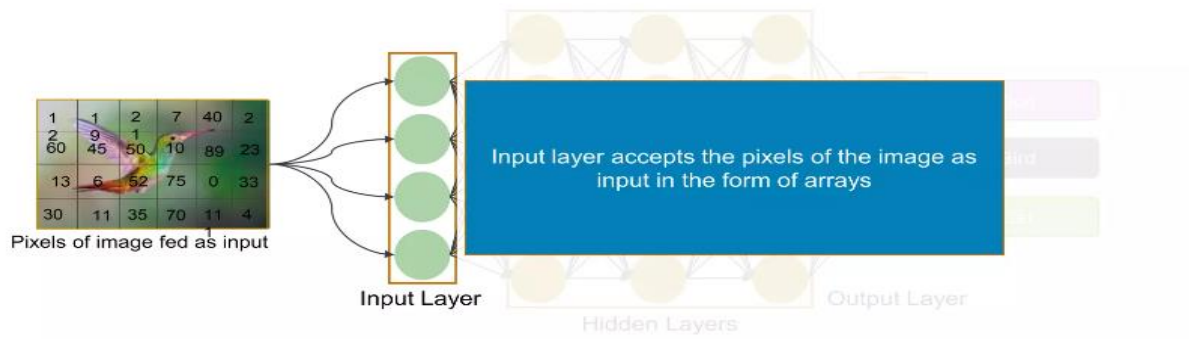
In CNN, ‘convolution’ is referred to as the mathematical function. It’s a type of linear operation in which you can multiply two functions to create a third function that expresses how one function’s shape can be changed by the other. In simple terms, two images that are represented in the form of two matrices, are multiplied to provide an output that is used to extract information from the image. CNN is similar to other neural networks, but because they use a sequence of convolutional layers, they add a layer of complexity to the equation. CNN cannot function without convolutional layers. CNN is a mathematical construct that is typically composed of three types of layers (or building blocks):

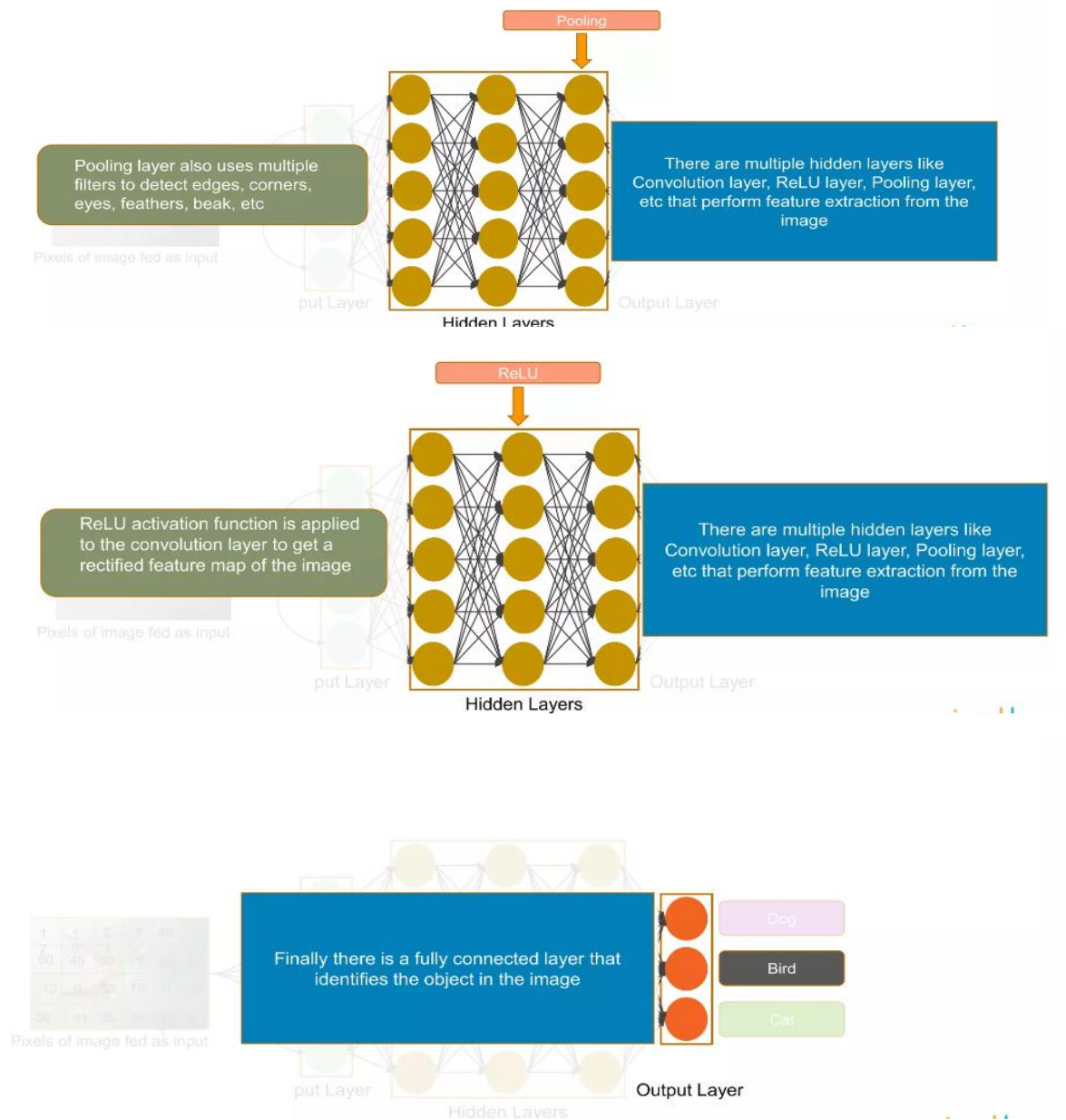
- i. convolution,**
- ii. pooling, and**
- iii. fully connected layers.**

The first two, convolution and pooling layers, perform feature extraction, whereas the third, a fully connected layer, maps the extracted features into final output, such as classification. A convolution layer plays a key role in CNN, which is composed of a stack of mathematical operations, such as convolution, a specialized type of linear operation.

How does CNN recognize images?

Consider the following images:





Convolutional Neural Network Architecture

Figure-1 shows the architecture for the convolutional neural network:

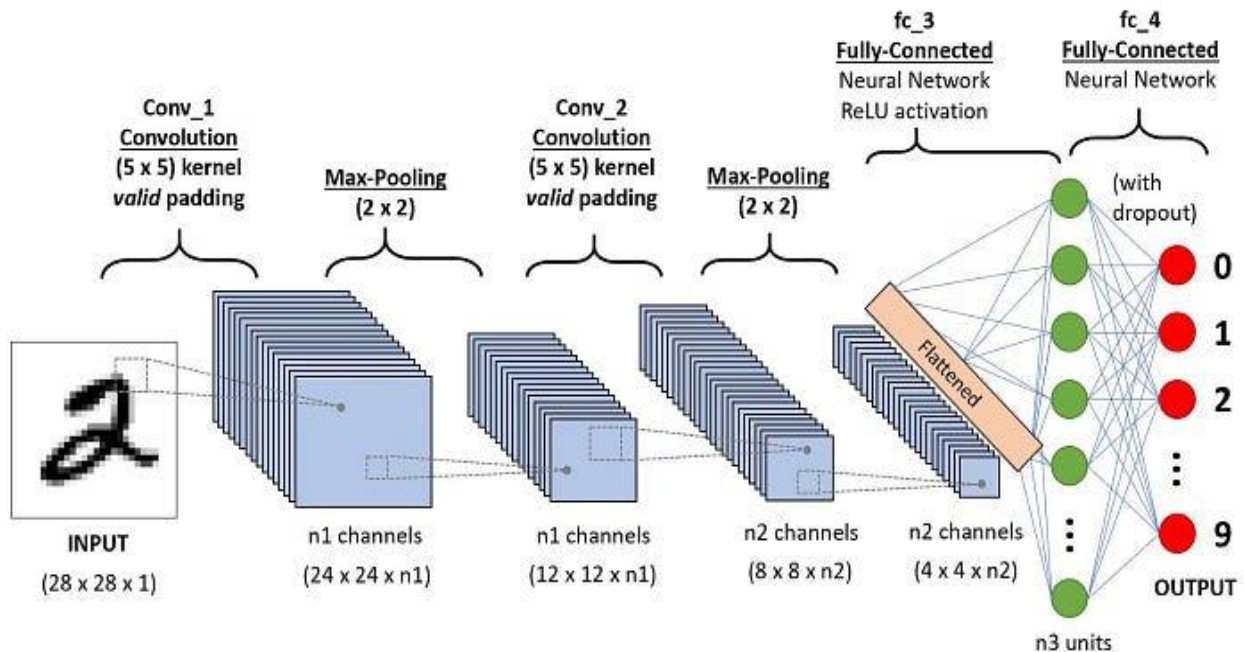


Figure-1: convolutional neural network architecture.

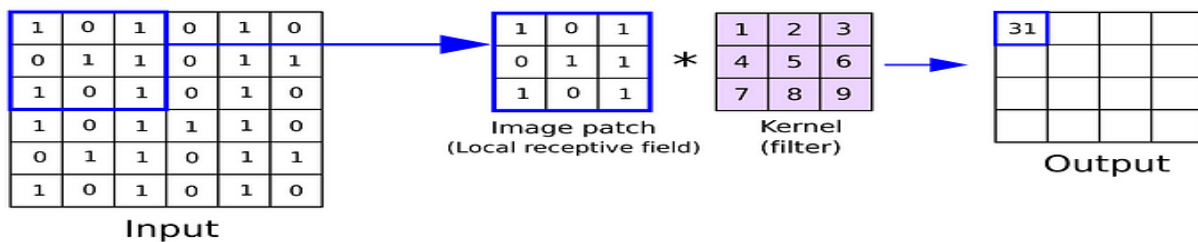
✚ Kernel or Filter or Feature detectors

In a convolutional neural network, the kernel is nothing but a filter that is used to extract the features from the images.

Formula = $[i - k] + 1$

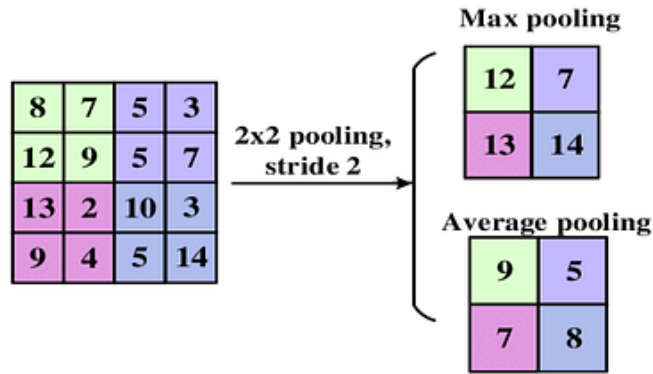
$i \rightarrow$ Size of input.

$k \rightarrow$ Size of kernel.



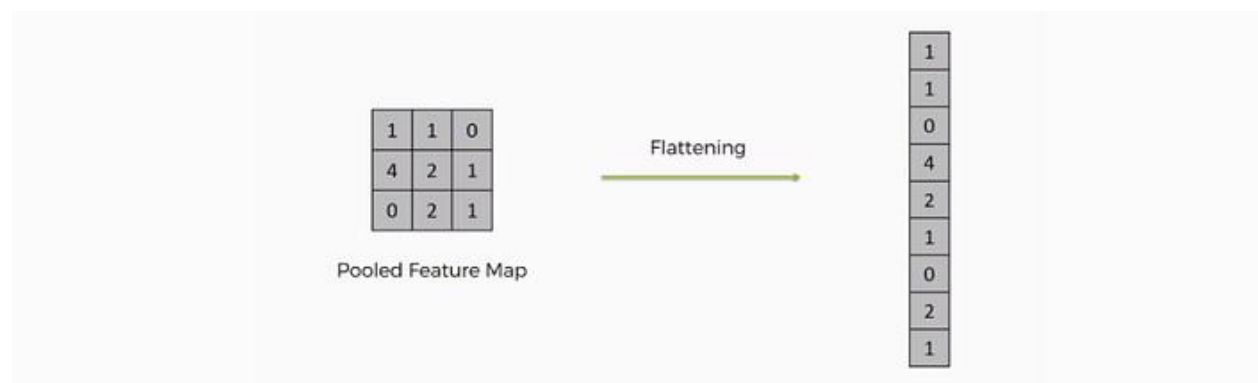
✚ Stride

Stride is a parameter of the neural network's filter that modifies the amount of movement over the image or video. we had stride 1 so it will take one by one. If we give stride 2 then it will take value by skipping the next 2 pixels.



✚ Flatten

Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector. The flattened matrix is fed as input to the fully connected layer to classify the image.



Layers used to build CNN

Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech, or audio signal inputs. They have three main types of layers, which are:

- ✓ **Convolutional layer.**
- ✓ **Pooling layer.**
- ✓ **Fully connected layer.**

Convolutional layer

This layer is the first layer that is used to extract the various features from the input images. In this layer, we use a filter or Kernel method to extract features from the input image.

$$\frac{W - F + 2P}{S} + 1$$

Pooling layer

The primary aim of this layer is to decrease the size of the convolved feature map to reduce computational costs. This is performed by decreasing the connections between layers and independently operating on each feature map. Depending upon the method used, there are several types of Pooling operations. We have Max pooling and average pooling.

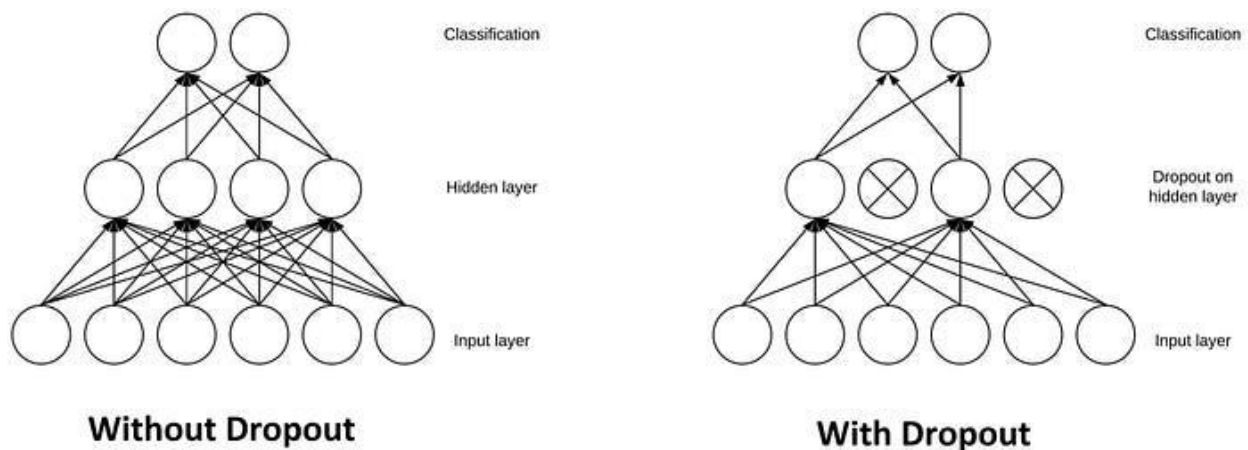
$$\frac{W - F}{S} + 1$$

Fully-connected layer

The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers. These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.

Dropout

Another typical characteristic of CNNs is a Dropout layer. The Dropout layer is a mask that nullifies the contribution of some neurons towards the next layer and leaves unmodified all others.



Activation Function

An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction. There are several commonly used activation functions such as

- **ReLU**
- **Softmax**
- **tanH and**

➤ Sigmoid functions

Each of these functions has a specific usage.

Sigmoid - The sigmoid non-linearity has the mathematical form $\sigma(\kappa) = 1/(1+e^{-\kappa})$. It takes a real-valued number and “squashes” it into a range between 0 and 1. However, a very undesirable property of sigmoid is that when the activation is at either tail, the gradient becomes almost zero. If the local gradient becomes very small, then in backpropagation it will effectively “kill” the gradient. Also, if the data coming into the neuron is always positive, then the output of sigmoid will be either all positives or all negatives, resulting in a zig-zag dynamic of gradient updates for weight.

tanH - The tanH function is very similar to the sigmoid function. The only difference is that it is symmetric around the origin. The range of values, in this case, is from -1 to 1.

Softmax - It is used in multinomial logistic regression and is often used as the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes.

ReLU - the main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time.

Designing a Convolutional Neural Network

we can build a convolutional neural network. We will be using Fashion-MNIST, which is a dataset of Zalando’s article images consisting of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28x28 grayscale image, associated with a label from 10 classes. The dataset can be downloaded from the link

https://www.researchgate.net/publication/319312259_Fashion-MNIST_a_Novel_Image_Dataset_for_Benchmarking_Machine_Learning_Algorithms

convolutional neural network has architecture as follows:

[INPUT]

→[CONV 1] → [BATCH NORM] → [ReLU] → [POOL 1]

→ [CONV 2] → [BATCH NORM] → [ReLU] → [POOL 2]

→ [FC LAYER] → [RESULT]

For both conv layers, we will use kernel of spatial size 5 x 5 with stride size 1 and padding of 2.

For both pooling layers, we will use max pool operation with kernel size 2, stride 2, and zero padding.

CONV 1
Input Size ($W_1 \times H_1 \times D_1$) = 28 x 28 x 1
• Requires four hyperparameter: ○ Number of kernels, $k = 16$ ○ Spatial extend of each one, $F = 5$ ○ Stride Size, $S = 1$ ○ Amount of zero padding, $P = 2$
• Outputting volume of $W_2 \times H_2 \times D_2$ ○ $W_2 = (28 - 5 + 2(2)) / 1 + 1 = 28$ ○ $H_2 = (28 - 5 + 2(2)) / 1 + 1 = 28$ ○ $D_2 = k$
Output of Conv 1 ($W_2 \times H_2 \times D_2$) = 28 x 28 x 16

Calculations for Conv1 Layer

POOL 1
Input Size ($W_2 \times H_2 \times D_2$) = 28 x 28 x 16
• Requires two hyperparameter: ○ Spatial extend of each one, $F = 2$ ○ Stride Size, $S = 2$
• Outputting volume of $W_3 \times H_3 \times D_2$ ○ $W_3 = (28 - 2) / 2 + 1 = 14$ ○ $H_3 = (28 - 2) / 2 + 1 = 14$
Output of Pool 1 ($W_3 \times H_3 \times D_2$) = 14 x 14 x 16

Calculations for Pool1 Layer

CONV 2
Input Size ($W_3 \times H_3 \times D_2$) = 14 x 14 x 16
• Requires four hyperparameter: ◦ Number of kernels, $k = 32$ ◦ Spatial extend of each one, $F = 5$ ◦ Stride Size, $S = 1$ ◦ Amount of zero padding, $P = 2$
• Outputting volume of $W_4 \times H_4 \times D_3$ ◦ $W_4 = (14 - 5 + 2(2)) / 1 + 1 = 14$ ◦ $H_4 = (14 - 5 + 2(2)) / 1 + 1 = 14$ ◦ $D_3 = k$
Output of Conv 2 ($W_4 \times H_4 \times D_3$) = 14 x 14 x 32

Calculations for Conv2 Layer

POOL 2
Input Size ($W_4 \times H_4 \times D_3$) = 14 x 14 x 32
• Requires two hyperparameter: ◦ Spatial extend of each one, $F = 2$ ◦ Stride Size, $S = 2$
• Outputting volume of $W_5 \times H_5 \times D_3$ ◦ $W_5 = (14 - 2) / 2 + 1 = 7$ ◦ $H_5 = (14 - 2) / 2 + 1 = 7$
Output of Pool 2 ($W_5 \times H_5 \times D_3$) = 7 x 7 x 32

Calculations for Pool2 Layer

FC Layer
Input Size ($W_5 \times H_5 \times D_3$) = 7 x 7 x 32
Output Size (Number of Classes) = 10

Size of Fully Connected Layer

MATLAB Problem for CNN

Problem Name: Face Recognition using CNN (Convolutional Neural Network)

```
%Matlab code - CNN for Face Recognition
faceDatasetPath = fullfile('FaceDataset');
imds = imageDatastore(faceDatasetPath,...
    'IncludeSubfolders',true,'LabelSource','foldernames');
figure;
perm = randperm(400,20);
for i = 1:20
    subplot(4,5,i);
    imshow(imds.Files{perm(i)});
end
numTrainFiles = 7;
[imdsTrain,imdsValidation] = splitEachLabel(imds,numTrainFiles,'randomize');

layers = [
    imageInputLayer([112 92 1])

    convolution2dLayer(3,12,'Padding','same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,20,'Padding','same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,'Stride',2)

    convolution2dLayer(3,64,'Padding','same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(40)
    softmaxLayer
    classificationLayer];

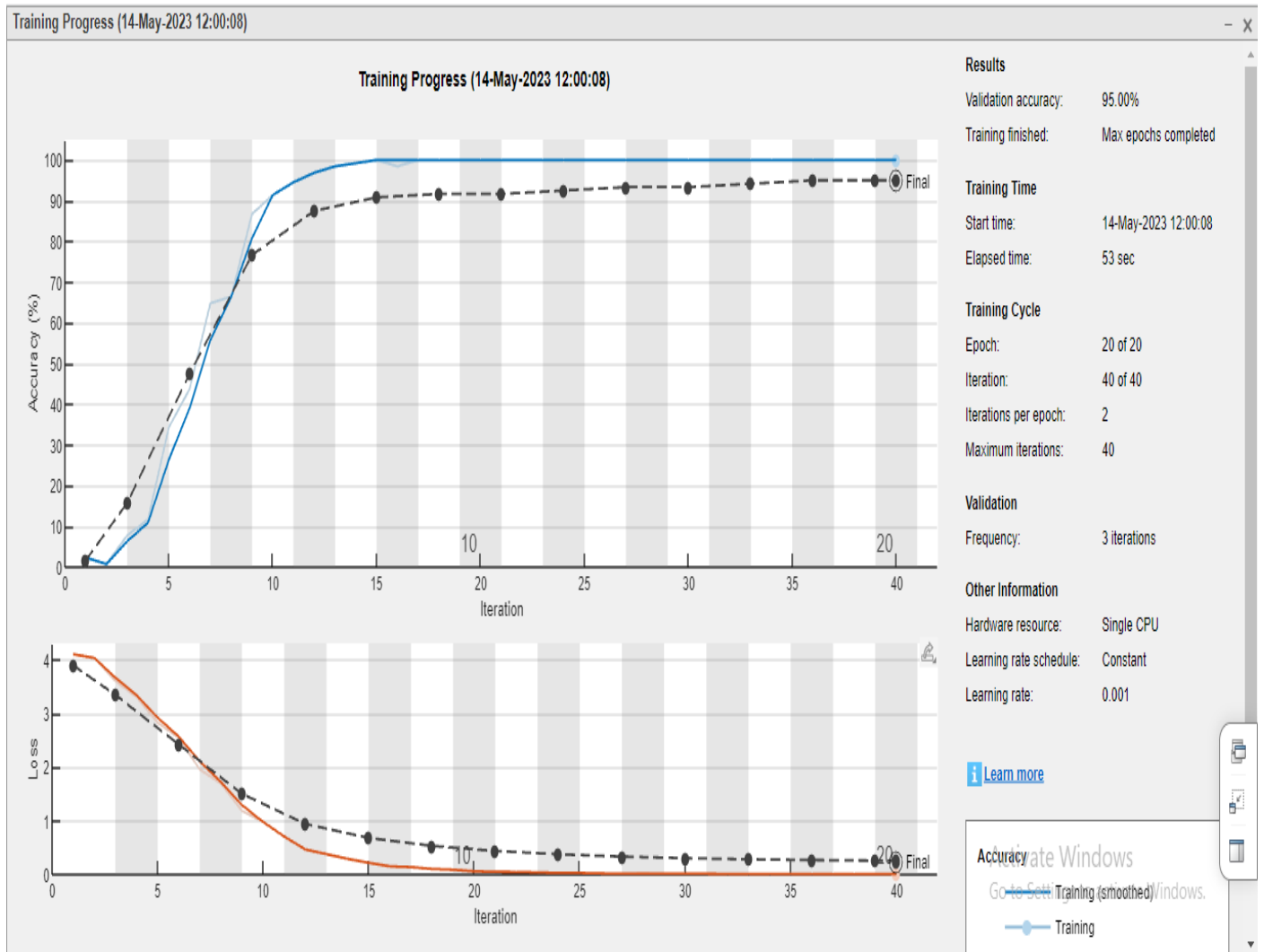
options = trainingOptions('sgdm', ...
    'InitialLearnRate',0.001, ...
    'MaxEpochs',20, ...
    'Shuffle','every-epoch', ...
    'ValidationData',imdsValidation, ...
    'ValidationFrequency',3, ...
    'Verbose',false, ...
    'Plots','training-progress');

net = trainNetwork(imdsTrain,layers,options);

YPred = classify(net,imdsValidation);
YValidation = imdsValidation.Labels;

accuracy = sum(YPred == YValidation)/numel(YValidation)
```

Output



The Dataset used for the MATLAB code is [ORL dataset(AT&T Dataset)] can be downloaded from the link and the details of the Dataset is also given

<https://www.kaggle.com/datasets/kasikrit/att-database-of-faces>

Training a Network

Training a network is a process of finding kernels in convolution layers and weights in fully connected layers which minimize differences between output predictions and given ground truth labels on a training dataset. Backpropagation algorithm is the method commonly used for training neural networks where loss function and gradient descent optimization algorithm play essential roles. A model performance under particular kernels and weights is calculated by a loss function through forward propagation on a training dataset, and learnable parameters, namely kernels and weights, are updated according to the loss value through an optimization algorithm called backpropagation and gradient descent, among others

Loss Function

A loss function, also referred to as a cost function, measures the compatibility between output predictions of the network through forward propagation and given ground truth labels. Commonly used loss function for multiclass classification is cross entropy, whereas mean squared error is typically applied to regression to continuous values. A type of loss function is one of the hyper parameters and needs to be determined according to the given tasks.

Applications

Below are some applications of Convolutional Neural Networks used today:

- ✓ **Medical Imaging:** CNNs can examine thousands of pathology reports to visually detect the presence or absence of cancer cells in images.
- ✓ **Audio Processing:** Keyword detection can be used in any device with a microphone to detect when a certain word or phrase is spoken (“Hey Siri!”). CNNs can accurately learn and detect the keyword while ignoring all other phrases regardless of the environment.
- ✓ **Object Detection:** Automated driving relies on CNNs to accurately detect the presence of a sign or other object and make decisions based on the output.
- ✓ **Synthetic Data Generation:** Using [Generative Adversarial Networks \(GANs\)](#), new images can be produced for use in deep learning applications including face recognition and automated driving.